CPSC370 - AGENTIC AI SYSTEMS · FINAL PROJECT

Chapman Agentic Scheduling Challenge

Project Overview

In this project, each team will design an LLM-based scheduling agent for one of Chapman University's 11 schools and colleges. Your agent will receive a set of available meeting times and classroom allocations from an instructor-provided university scheduling coordinator. Using those resources, your agent must create a feasible schedule for its academic unit, explain its choices, respond to proposed schedule changes, and repair its schedule when resources or constraints change.

All 11 academic-unit agents will be integrated at the end of the semester with a common coordinator. The final system will therefore be a class-built multi-agent scheduling environment: each unit controls its own local scheduling decisions while the coordinator manages university-wide conflicts and requests changes.

The project is intentionally designed so that strong programming is not required. You may use any contemporary LLM agent framework, a simple Python wrapper, prompt/configuration files, retrieval over historical schedule data, or a combination of approaches. What matters is whether your agent behaves correctly, uses evidence and constraints appropriately, and conforms exactly to the common interface described in this specification.

Academic Units

Each team will be assigned one of the following academic units:

1. Argyros College of Business and Economics
2. Attallah College of Educational Studies
3. College of Performing Arts
4. Crean College of Health and Behavioral Sciences
5. Dodge College of Film and Media Arts
6. Fowler School of Engineering
7. Fowler School of Law
8. Schmid College of Science and Technology
9. School of Communication
10. School of Pharmacy

11. Wilkinson College of Arts, Humanities, and Social Sciences

The 11-unit list above follows Chapman's current schools-and-colleges structure. Your team represents the academic unit as a whole, not a single department.

What the Coordinator Provides

Your agent does not choose the university-wide room inventory or time grid. The coordinator provides the resource envelope your academic unit may use.

- A list of courses/sections that your academic unit must schedule.
- A list of available meeting times.
- A list of classrooms allocated or available to your unit.
- Room capacity and relevant room features.
- Current faculty/course constraints supplied for the scheduling run.
- Any university-wide hard constraints that apply to all agents.

What Your Agent Decides

- Which course is assigned to which available time.
- Which allocated classroom is assigned to each course.
- How to resolve competing preferences within your unit.
- Which proposed schedule changes are acceptable.
- Which alternatives to suggest when a proposed change is unacceptable.
- How to repair the unit schedule when a room, time, faculty availability, or other resource changes.

Use of Historical Schedule Data

You may use historical Chapman schedule data in any way you believe improves your agent. Historical data is optional and advisory; it must never override current hard constraints.

- Use prior schedules directly as examples in the agent prompt.
- Retrieve similar historical courses or schedules at runtime.
- Compute empirical preferences, such as common meeting days or times for a course or faculty member.

- Infer common room types or scheduling patterns.
- Use historical schedules to construct few-shot examples.
- Ignore historical data entirely and rely on explicit current constraints.

If you use historical data, document exactly how it is used and identify any limitations. Copying a previous semester's schedule without reasoning about the current resource allocation is not sufficient.

Scheduling Rules

Your agent must distinguish between hard constraints and soft preferences. Hard constraints determine whether a schedule is valid. Soft preferences determine whether one valid schedule is better than another.

HARD CONSTRAINTS

- A course must use one of the time slots supplied by the coordinator.
- A course must use one of the rooms supplied or allocated to the unit.
- Two courses may not occupy the same room at the same time.
- One instructor may not teach two courses at the same time.
- Expected enrollment may not exceed room capacity.
- A course requiring a specialized room feature must be assigned to a room with that feature.
- Explicit course-pair conflict constraints supplied in the input must be respected.
- Any additional hard constraints included in the request must be respected.

A response containing an avoidable hard-constraint violation is considered invalid, even if the schedule is otherwise attractive.

SOFT PREFERENCES

Soft preferences may differ by academic unit and may be informed by historical schedules. Examples include:

- Faculty preferred teaching days or times.
- Avoiding early morning, late evening, or Friday sections.
- Keeping required courses from the same program separated.
- Minimizing long gaps in a faculty member's teaching day.
- Using historically common meeting patterns.
- Keeping a course in a preferred building or room type.
- Minimizing changes when repairing an existing schedule.

Your team decides how to represent and prioritize soft preferences. You may use a weighted scoring function, LLM reasoning, rules, an optimizer, or a hybrid approach. You must be able to explain your choice.

Required Agent Capabilities

Every academic-unit agent must support all three operations below.

Operation	Purpose	Required behavior
`generate_schedule`	Create a schedule from the supplied resource envelope.	Return a complete proposed local schedule plus rationale and unresolved issues.
`evaluate_change`	Evaluate a proposed change to one scheduled course.	Accept or reject the change, explain why, and provide valid alternatives when rejecting.
`repair_schedule`	Repair an existing schedule after a disruption.	Produce a new valid schedule while minimizing unnecessary changes where possible.

Technical Freedom

You may implement the internals of your agent however you want, provided it follows the required interface. Examples include:

- A simple Python program that calls an LLM.
- OpenAI Agents SDK, LangGraph, CrewAI, PydanticAI, Google ADK, Microsoft Agent Framework, or another agent framework.
- A prompt-driven agent with configuration files.
- Retrieval-Augmented Generation (RAG) over historical schedules or policy documents.
- A hybrid system combining deterministic constraint checks with LLM reasoning.
- A mathematical optimizer used as a tool by an LLM agent.

Using a sophisticated framework does not automatically earn a higher grade. A simple system that is reliable, explainable, and well evaluated may outperform a complex system.

Required Integration Interface

All teams must conform to the same interface so the university coordinator can call every academic-unit agent without knowing how it is implemented.

REQUIRED PYTHON ENTRYPOINT

Your submission must expose the following callable from `agent.py`:

```
def run_agent(request: dict) -> dict:
    """Process one coordinator request and return a JSON-compatible dictionary."""
    ...
```

The instructor starter code will handle model credentials and basic launching/integration. Teams using a framework may wrap their framework inside `run_agent()`. Teams using a prompt/configuration approach may use the provided starter implementation and primarily edit prompts, constraints, and supporting data.

COMMON REQUEST ENVELOPE

```
{
  "protocol_version": "1.0",
  "request_id": "req-001",
  "operation": "generate_schedule",
  "unit": "Fowler School of Engineering",
  "semester": "Fall 2027",
  "payload": { ... }
}
```

Required fields:

- **protocol_version**: must be "1.0".
- **request_id**: unique identifier supplied by the coordinator; return it unchanged.
- **operation**: one of generate_schedule, evaluate_change, or repair_schedule.
- **unit**: the academic unit your agent represents.
- **semester**: scheduling context.
- **payload**: operation-specific input described below.

COMMON RESPONSE ENVELOPE

```
{
  "protocol_version": "1.0",
  "request_id": "req-001",
  "unit": "Fowler School of Engineering",
  "status": "completed",
  "result": { ... },
  "warnings": [],
  "errors": []
}
```

- **status** must be completed, partial, or failed.
- **result** contains operation-specific output.
- **warnings** contains non-fatal issues or uncertainty.
- **errors** contains fatal problems that prevented completion.
- The response must be JSON-serializable. Do not return framework-specific objects.

Operation 1: generate_schedule

Request payload:

```
{
  "courses": [
    {
      "course_id": "CPSC-350-01",
      "title": "Data Structures and Algorithms",
      "instructor_id": "FAC-014",
      "expected_enrollment": 34,
      "required_features": ["computer"],
      "preferred_times": ["MW-1030", "TR-1000"],
      "avoid_times": ["MW-0800"],
      "conflicts_with": ["CPSC-380-01"]
    }
  ],
  "timeslots": [
    {
      "timeslot_id": "MW-1030",
      "days": ["M", "W"],
      "start": "10:30",
      "end": "11:45"
    }
  ],
  "rooms": [
    {
      "room_id": "SWE-150",
      "capacity": 40,
      "features": ["computer"]
    }
  ],
  "faculty_constraints": [
    {
      "instructor_id": "FAC-014",
      "unavailable_times": ["TR-1300"],
      "preferred_times": ["MW-1030"]
    }
  ],
  "global_constraints": [],
  "context": {}
}
```


Required result:

```
{
  "schedule": [
    {
      "course_id": "CPSC-350-01",
      "timeslot_id": "MW-1030",
      "room_id": "SWE-150",
      "rationale": [
        "Room capacity satisfies expected enrollment.",
        "Room provides required computing feature.",
        "Time is preferred by the instructor."
      ],
      "preference_score": 0.91
    }
  ],
  "unresolved": [],
  "summary": "All courses scheduled with no hard-constraint violations."
}
```

`preference_score` must be a number from 0.0 to 1.0 representing how strongly the agent prefers the selected assignment relative to other feasible choices. You decide how to compute or estimate this score, but document your method.

Operation 2: evaluate_change

The coordinator uses this operation when it needs your unit to consider moving one course because of a university-wide conflict.

```
{
  "current_schedule": [ ... ],
  "proposed_change": {
    "course_id": "CPSC-350-01",
    "new_timeslot_id": "TR-1000",
    "new_room_id": "SWE-150"
  },
  "reason": "Resolve conflict with another academic unit."
}
```

Required result:

```
{
  "decision": "reject",
  "reason": "The proposed time conflicts with another required course.",
  "impact_score": 0.82,
  "alternatives": [
    {
      "timeslot_id": "MW-1300",
      "room_id": "SWE-150",
      "impact_score": 0.22,
      "reason": "No hard conflicts and only a minor preference penalty."
    }
  ]
}
```

- **decision** must be accept or reject.
- **impact_score** must be between 0.0 and 1.0, where 0.0 means essentially no disruption and 1.0 means severe disruption or infeasibility.
- If you reject a proposal and at least one valid alternative exists, return one or more alternatives.
- A rejection must identify the specific constraint or preference that caused the rejection.

Operation 3: repair_schedule

The coordinator uses this operation after a disruption, such as a room becoming unavailable, a faculty constraint changing, or enrollment increasing.

```
{
  "current_schedule": [ ... ],
  "disruptions": [
    {
      "type": "room_unavailable",
      "room_id": "SWE-150"
    }
  ],
  "timeslots": [ ... ],
  "rooms": [ ... ],
  "courses": [ ... ],
  "faculty_constraints": [ ... ]
}
```

Required result:

```
{
  "schedule": [ ... ],
  "changes": [
    {
      "course_id": "CPSC-350-01",
      "old_timeslot_id": "MW-1030",
      "old_room_id": "SWE-150",
      "new_timeslot_id": "MW-1030",
      "new_room_id": "SWE-201",
      "reason": "Original room unavailable."
    }
  ],
  "change_count": 1,
  "unresolved": [],
  "summary": "Schedule repaired with one course moved."
}
```

The repair objective is not simply to produce any new schedule. Your agent should preserve as much of the existing schedule as possible while restoring feasibility.

Error Handling Requirements

- Never invent a room or time slot that is absent from the coordinator input.
- Never silently ignore an unschedulable course.
- If a complete valid schedule is impossible, return status = partial and list the unscheduled courses in unresolved.
- If the request is malformed or your agent cannot operate, return status = failed and explain the error.
- Do not crash the integration process because one course cannot be scheduled.

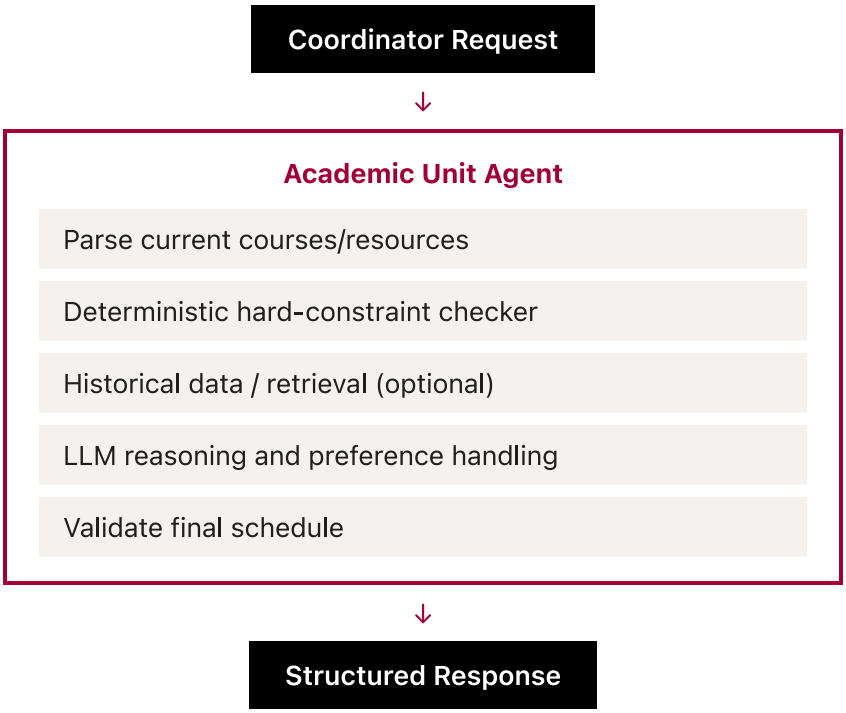
Agent Design Requirements

Your agent must include all of the following.

Requirement	Expectation
LLM-based reasoning	An LLM must play a substantive role in scheduling, evaluating changes, explaining decisions, or coordinating tools.
Explicit constraint handling	Your system must have a reliable method for checking hard constraints. Purely asking an LLM to 'be careful' is not enough.
Structured output	All coordinator responses must conform to the JSON-compatible interface in this specification.
Rationale	The agent must explain important scheduling decisions in concise, specific terms.
Alternative generation	When rejecting a feasible-to-repair change, the agent should propose alternatives rather than only saying no.
Historical-data decision	Document whether and how you use historical schedule data.
Evaluation	Test your agent systematically before final integration.

Recommended Architecture

A strong solution will normally separate deterministic constraint checking from LLM judgment. For example:



This is a recommendation, not a required implementation. The critical requirement is that hard constraints are checked reliably.

Testing and Evaluation

Each team must create at least 12 test cases. At least four must test generate_schedule, four must test evaluate_change, and four must test repair_schedule.

- At least 3 tests must intentionally create difficult or impossible conditions.
- At least 2 tests must check room capacity or room-feature constraints.
- At least 2 tests must check faculty or course conflicts.
- At least 2 tests must evaluate whether the agent proposes sensible alternatives.
- At least 1 test must use a disruption that should be repairable with only one or two changes.

REQUIRED EVALUATION METRICS

Metric	Definition	Target
Hard-constraint compliance	Percentage of completed test schedules containing zero hard violations.	**100%**
Task completion	Percentage of test cases for which the agent returns a usable response.	**At least 90%**
Interface compliance	Percentage of responses that conform to the required schema.	**100%**
Alternative validity	Percentage of proposed alternatives that are actually feasible.	**At least 90%**
Repair stability	Average number of unnecessary course changes during repair.	**Lower is better**

Project Milestones

Milestone 1 — **Unit Model:** Course representation, constraints, preferences, historical-data plan, and proposed agent architecture.

Milestone 2 — **Standalone Agent:** Agent can successfully complete generate_schedule for a supplied practice case.

Milestone 3 — **Change Negotiation:** Agent supports evaluate_change and proposes valid alternatives.

Milestone 4 — **Repair and Evaluation:** Agent supports repair_schedule and has a completed 12-case test suite with metrics.

Milestone 5 — **Integration Readiness:** Agent passes the instructor interface-compliance test and runs through the common wrapper.

Final — **Final Integration:** All 11 agents are connected to the university coordinator and participate in a university-wide scheduling and disruption exercise.

Final Integration Exercise

On final integration day, the coordinator will provide each unit with its allocated rooms and time slots and request a schedule. The coordinator will then identify university-wide issues and ask individual academic-unit agents to evaluate changes. After the initial schedule converges, one or more disruptions will be introduced.

Possible disruptions include:

- A classroom becomes unavailable.
- A room's capacity changes.
- A faculty member becomes unavailable during a previously valid time.
- Enrollment in a course increases.
- A new university-wide blackout period is introduced.
- A course must be moved with minimal disruption to the rest of the schedule.

Your grade does not depend on the entire class-wide system producing a perfect university schedule. You are graded primarily on the quality, reliability, interface compliance, and evaluation of your academic-unit agent.

Required Deliverables

- `agent.py` exposing `run_agent(request: dict) -> dict`.
- All prompts, configuration files, and supporting code needed to run the agent.
- Any historical data or derived data used by the agent, or clear instructions for accessing it.
- README.md with setup and execution instructions.
- agent_design.pdf or agent_design.md (2-4 pages) describing architecture, constraints, use of historical data, and major design decisions.
- tests/ containing at least 12 documented test cases.
- evaluation.csv or evaluation.json containing your test results and required metrics.
- A short final reflection describing one failure mode you discovered and how you addressed it.

SUBMISSION DIRECTORY

```
your_unit_name/
  agent.py
  README.md
  prompts/
  config/
  data/
  tests/
  agent_design.pdf (or .md)
  evaluation.csv (or .json)
```

Grading

Component	Percent	What matters
Academic-unit model and constraints	**15%**	Accurate representation of courses, resources, hard constraints, and meaningful soft preferences.
Agent design and LLM use	**15%**	Clear role for the LLM; appropriate architecture; design is justified rather than unnecessarily complex.
Schedule generation	**15%**	Produces feasible local schedules and handles unschedulable cases correctly.
Change evaluation and negotiation	**15%**	Accepts/rejects changes correctly, explains impacts, and proposes useful alternatives.
Repair behavior	**10%**	Repairs disruptions while preserving as much of the existing schedule as practical.
Testing and evaluation	**15%**	Thoughtful test cases, quantitative metrics, and evidence of iteration.
Interface and integration compliance	**10%**	Exact adherence to the common interface and successful coordinator integration.
Documentation and final explanation	**5%**	Clear README/design explanation and concise reflection on limitations.

Important Design Principles

- A more complex agent is not automatically a better agent.
- Hard constraints must be validated, not merely described in a prompt.
- Historical schedules are evidence of prior practice, not authoritative requirements.
- A scheduling agent should be able to explain why it made important choices.
- When a request cannot be satisfied, a useful agent explains the problem and offers alternatives.
- The final integrated system is multi-agent because scheduling knowledge and decision authority are distributed across academic units.

Academic Integrity and External Services

You may use contemporary LLM APIs, agent frameworks, coding assistants, and open-source libraries. You are responsible for understanding and being able to explain your submitted system. Do not place confidential or non-public university information into external services. Use only data you are authorized to use. If you use historical schedules, document their source.